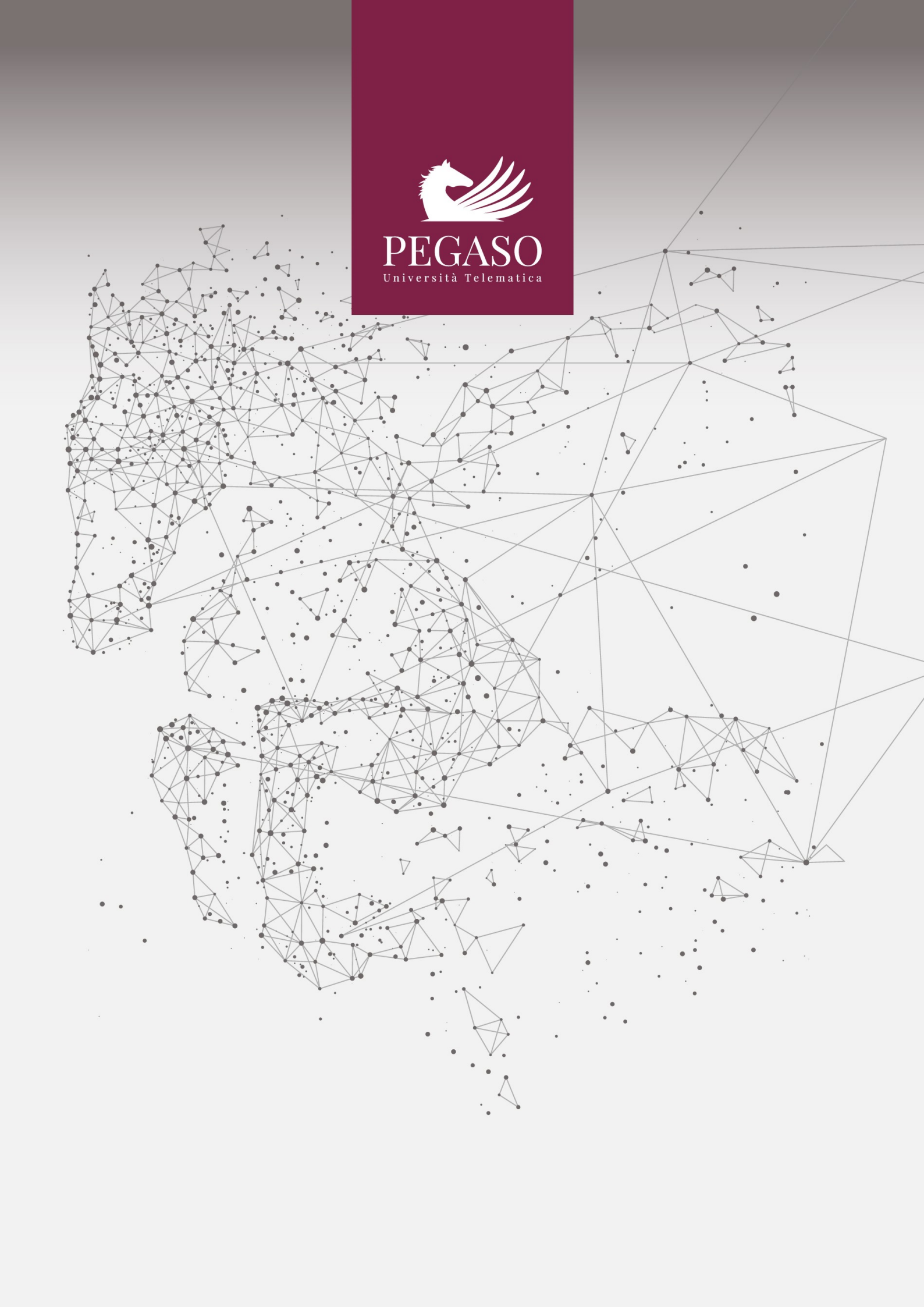

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# Indice

**1. IDENTIFICATORI ..... 3**
**2. TIPI DI DATI PRIMITIVI ..... 6**
**3. TIPI DI DATI COMPLESSI: OGGETTI ..... 8**
**BIBLIOGRAFIA ..... 14**

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 1. Identificatori

Mediante gli identificatori si attribuiscono nomi a metodi, classi, oggetti, package, variabili e costanti e anche ad altri elementi della programmazione quali interfacce, enumerazioni, record e annotazioni.

Al di là delle specifiche regole definite da Java, occorre in generale prestare molta attenzione sulla scelta degli identificatori.

Assegnare dei nomi nella programmazione significa associare un nome ad una certa astrazione, cosa particolarmente importante specie nella programmazione ad oggetti dove le astrazioni in genere riguardano anche concetti di alto livello vicini al mondo reale che ci circonda. Scegliere un identificatore potrebbe sembrare una banalità ma non lo è, e non deve mai esserlo: bisogno porre molta attenzione per mantenere un codice comprensibile e chiaro in ogni suo punto, nel tempo in ogni fase di sviluppo e del ciclo di vita del software (come ad esempio in una successiva fase di manutenzione o di upgrade) e per qualsiasi altra persona che si dovesse avvicendare nello sviluppo. A volte scegliere un identificatore corretto richiede diversi minuti ed è molto frequente che si decida di modificare un identificatore in un secondo momento. Quando un identificatore è difficile da scegliere, probabilmente è un segno che abbiamo le idee chiare su cosa dobbiamo fare.

Detto questo come linea guida generale, per non incorrere in errori di compilazione, nella scelta degli identificatori occorre seguire delle semplici regole.

Per creare un identificatore si può usare solo un certo insieme di caratteri, numeri e simboli. In particolare, in un identificatore:

il primo carattere può essere A-Z, a-z, _, $, il secondo e i successivi possono essere A-Z, a-z, _, $, 0-9.

Quindi a2 è un identificatore valido, 2a non lo è.

Un identificatore non può essere una parola definita dal linguaggio Java perché nell'ambito del linguaggio hanno già un preciso significato. Tra queste parole la maggior parte sono parole chiave. Nella tabella in Fig. 1 sono mostrate le parole definite da Java (in ordine alfabetico) che non possono quindi essere usate come identificatori. Le parole iniziano tutte con lettera minuscola.

Alcune le conosciamo già come class, static, o return per esempio. Altre saranno note al momento in cui si approfondisce la conoscenza del linguaggio.

In base alla tabella non potremo quindi chiamare una variabile class, oppure un metodo void.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

Tra le parole riservate ci sono anche goto e const, che in realtà non hanno significato in Java ma che sono state inserito perché non possono essere usati come identificatori, sono dette parole riservate. Si noti anche @interface iniziando con il simbolo @ non potrebbe comunque essere usata come identificatore. Infatti tali simboli non sono validi per definire un identificatore.

| abstract | assert | boolean | break | byte |
|---|---|---|---|---|
| case | catch | char | class | const |
| continue | default | do | double | else |
| enum | extends | false | final | finally |
| float | for | goto | if | implements |
| import | instanceof | int | interface | long |
| native | new | null | package | private |
| protected | public | return | short | static |
| strictfp | super | switch | synchronized | this |
| throw | throws | transient | true | try |
| void | volatile | while | enum | @interface |

**Fig. 1: Parole riservate in Java**

Al di là delle regole appena descritte esistono anche direttive standard sui nomi fornite direttamente da Oracle (e originariamente da Sun Microsystems) per raggiungere uno standard anche nello stile di implementazione.

Trattando di astrazione, la prima raccomandazione che abbiamo fatto è stata quella di scegliere degli identificatori che fossero significativi nel contesto del programma (ovvero del problema che stiamo risolvendo). Infatti se scriviamo un programma usando la classe a, che definisce le variabili b,c,d e il metodo e, sicuramente ridurremmo la comprensibilità del programma stesso.

A tale scopo può essere utile avere dei nomi che sono composti da più parole unite a formare un unico identificatore. Si può fare e anzi è consigliabile. In tal caso si iniziare ciascuna parola con la lettera maiuscola.

Ad esempio se stiamo scrivendo un programma per la gestione degli ingressi e partecipazioni ad una conferenza risultano più chiari identificatori tipo ad esempio numeroPartecipanti.

Non c'è una convenzione sul numero di caratteri massimo da usare per un identificatore ma in Java è meglio preferire la chiarezza alla brevità di un nome. Naturalmente con un po' di buon senso.

Esistono delle convenzioni specifiche per gli identificatori a seconda del tipo di componente a cui bisogna assegnare un nome.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

### a. Variabili per classi, variabili, metodi, costanti e package

Un identificatore di una classe deve iniziare con una lettera maiuscola e deve essere un unico elemento. Esempi di identficatori per una classe potrebbero essere: Persona, MacchinaDaCorsa, FiguraGeometrica

Un identificatore di una variabile deve sempre iniziare con una lettera minuscola. Se l'identificatore di una variabile è composto da più parole si procede come per le classi (tranne per il fatto che la prima lettera deve essere sempre minuscola) ovvero si compongono le parole in un unico identificatore e facendo iniziale ognuna di esse con lettera maiuscola. Esempi di identificatori per una variabile potrebbero essere: pesoSpecifico, numeroDiMinutiTotale, x, i, numeroLati.

Gli identificatori delle costanti invece si devono distinguere nettamente dagli altri e tutte le lettere dovranno essere maiuscole. Se l'identificatore è composto da più parole, queste si separano con un underscore. Ad esempio: NUMERO_LATI, PI_GRECO.

Per un identificatore di un metodo, valgono le stesse regole per gli identificatori delle variabili: iniziano per lettera minuscola e l'identificatore può essere composto da più parole. Un identificatore di un metodo si distinguerà da quello di una variabile per il fatto che sarà sempre seguito dalle parentesi tonde. Inoltre è buona norma che l'identificatore di un metodo contenga un verbo.

Gli identficatori dei metodi solitamente conterranno verbi, ad esempio stampaNumeroPartecipantiAlWorkshop o somma.

Esempi di identificatori validi per un metodo saranno: sommaDueNumeri(int a, int b), cercaUnaParola (String parola), stampa().

Gli identificatori dei package contengono generalmente solo lettere minuscole. Se abbiamo bisogno di identificatori composti da più parole è possibile creare package innestati sfruttano la relazione gerarchica tra le cartelle. Per esempio, se volessimo creare un package di assistenza clienti, è possibile creare due package innestati nel seguente modo: assistenza.clienti ovvero un package assistenza con all'interno un package clienti.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 2. Tipi di dati primitivi

Java definisce 8 tipi di dati primitivi, ovvero tipi di dati semplici che differenzieremo dai tipi di dati complessi riconducibili agli oggetti. I tipi di dati primitivi possono essere classificati in 4 categorie:

- tipi interi
- tipi floating point
- tipo testuale
- tipo logico-booleano

I **tipi di dati interi** sono quattro: byte, short, int e long e vengono memorizzati tramite il metodo del complemento a due.

Differiscono per la lunghezza dei bit e quindi per l'intervallo dei numeri rappresentanti. Infatti un byte può immagazzinare un intero. Un byte può rappresentare un intero utilizzando un byte (8bit) e quindi da -128 a 127, uno short usa 16 bit, in int usa 32 bit e il long usa 64 bit. In generale, l'intervallo dei numeri rappresentati va da $-2^{(n-1)}$ a $2^n$. In Java non esistono tipi senza segno (unsigned).

Come mostrato negli esempi di Fig. 2, per immagazzinare un intero in Java è possibile usare quattro tipi di sistemi numerici: decimale, binaria, ottale e esadecimale. Per distinguere queste notazioni tra loro e dalla notazione decimale si usa anteporre 0b per la notazione binaria, 0 per la notazione ottale, 0x per la notazione esadecimale.

```
byte b=10;           //notazione decimale: b vale 10
short s = 022;       // notazione ottale: s vale 18
long l = 0x12aCd;    // notazione esadecimale: l vale 74493
int i= 1000000000;   //notazione decimale: i vale 1000000000
int n = 0b10100001010001011010000101000101
                     //notazione binaria: n vale -1589272251
```

**Fig. 2: Uso dei tipi di dati interi**

I **tipi di dati floating point** servono per memorizzare numeri decimali. Java definisce due tipi a virgola mobile: float (a 32 bit) e double (64 bit).

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

Java utilizza per i valore flaoting point (a virgola mobile) lo standard di decodifica IEEE-751.

Oltre alla notazione classica double d = 0.0126; si può usare in alternativa la notazione esponenziale: double d = 1.26E-2;

Il **tipo di dato char** permette di immagazzinare il singolo carattere. Si tratta di un tipo di dato non particolarmente usato perché nella maggior parte dei casi è necessario usare stringhe che consentono di immagazzinare sequenze di caratteri. Il tipo primitivo char è immagazzinato in 16 bit e può definire ben 65536 caratteri diversi. La codifica Unicode si occupa di standardizzare tutti caratteri.

Il char memorizza un qualsiasi carattere che si trova sulla tastiera oppure un valore Unicode in esadecimale che identifica univocamente un determinato carattere anteponendo il prefisso \u. Inoltre il char può immagazzinare caratteri di escape speciali, ossia sequenze di caratteri che corrispondono alla pressione su un tasto della tastiera che non è direttamente rappresentato da un carattere o un simbolo, ma che provoca particolari comportamenti nella stampa. Fra questi ad esempio c'è \n che equivale a line feed. In tutti i casi occorre comprendere tra apici singoli il valore da assegnare.

Il **tipo di dato boolean** utilizza solo un bit per momorizzare un valore e gli unici valori che può immagazzinare sono true e false. Il tipo boolean serve per effettuare scelte che condizionano l'esecuzione del programma.

In Fig. 3 è mostrata una tabella riepilogativa sui tipi di dati primitivi.

| Nome tipo | Tipo di valore | Memoria usata | Valori consentiti |
|---|---|---|---|
| `byte` | Intero | 1 byte | da -128 a 127 |
| `short` | Intero | 2 byte | da -32.768 a 32.767 |
| `int` | Intero | 4 byte | da -2.147.483.648<br>a 2.147.483.647 |
| `long` | Intero | 8 byte | da -9.223.372.036.854.775.808<br>a 9.223.372.036.854.775.807 |
| `float` | Numero in virgola mobile (precisione singola) | 4 byte | da $-3.40282347 \times 10^{38}$<br>a $3.40282347 \times 10^{38}$ |
| `double` | Numero in virgola mobile (precisione doppia) | 8 byte | da $-1.7*10^{308}$<br>a $1.7*10^{308}$ |
| `char` | Carattere singolo | 2 byte | Tutti i caratteri della codifica Unicode (65.535 caratteri) |
| `boolean` | Valore di verità | 1 bit | `true` o `false` |

**Fig. 3: Tabella riepilogativa sui tipi di dati primitive**

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 3. Tipi di dati complessi: Oggetti

Abbiamo già visto come istanziare oggetti di una certa classe. Dobbiamo prima dichiarare un oggetto di tale classe con una sintassi del tipo:

NomeClasse nomeOggetto; //dichiarazione

Per poi istanziarlo con la parola chiave new:

nomeOggetto=new NomeClasse(); //istanza e assegnazione

Nel secondo passo viene istanziato un oggetto invocando un costruttore mediante l'operatore new, che puoi viene assegnato a nomeOggetto.

Concentrandosi sul primo passo possiamo vedere come la dichiarazione di un oggetto è del tutto simile a dichiarare un tipo di dato primitivo.

La variabile che identifica un oggetto infatti è una particolare variabile detta reference. Ma non è una variabile di tipo primitivo perché è un tipo definito da una classe (con al suo interno un certo numero di variabili e metodi). Inoltre c'è un'altra sottile differenza tra una variabile di un tipo primitivo e uno complesso e risiede nel modo in cui si memorizza il valore.

La differenza può essere resa evidente con un esempio, partendo dalla definizione di una classe che astrae in maniera banale il concetto di data (Data) e creando poi un'istanza chiamata unGiorno. Poi consideriamo come tipo di dato primitivo un double chiamato unNumero a cui diamo valore 5.0 (vedi Fig. 4).

```
…
public class Data {
public int giorno;
public int mese;
public int anno;
}
…
Data unGiorno = new Data();
…
double unNumero = 5.0;
```

**Fig. 4: Esempio di confronto tra uso di tipi di dati primitivi e complessi**

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

Potrebbe aiutare una schematizzazione di come sono rappresentati i dati in memoria quando questa applicazione è in esecuzione (vedi Fig. 5).

In pratica una variabile di tipo primitivo ha come valore il valore memorizzato della variabile. Una variabile di tipo non primitivo che si riferisce ad un oggetto corrisponde ad una area di memoria dove è presente un indirizzo numerico che individua un'altra area di memoria dove è memorizzato l'oggetto. Come si può vedere dalla Fig. 5 nel caso di unNumero si ha una cella di memoria denominata con identificatore unNumero in cui si trova il valore 5.0. Nel caso dell'oggetto si una cella di memoria con identificatore unGiorno che contiene un indirizzo di memoria a partire dal quale si troveranno le tre celle di memoria in cui si trovano gli attributi giorno, mese, anno.

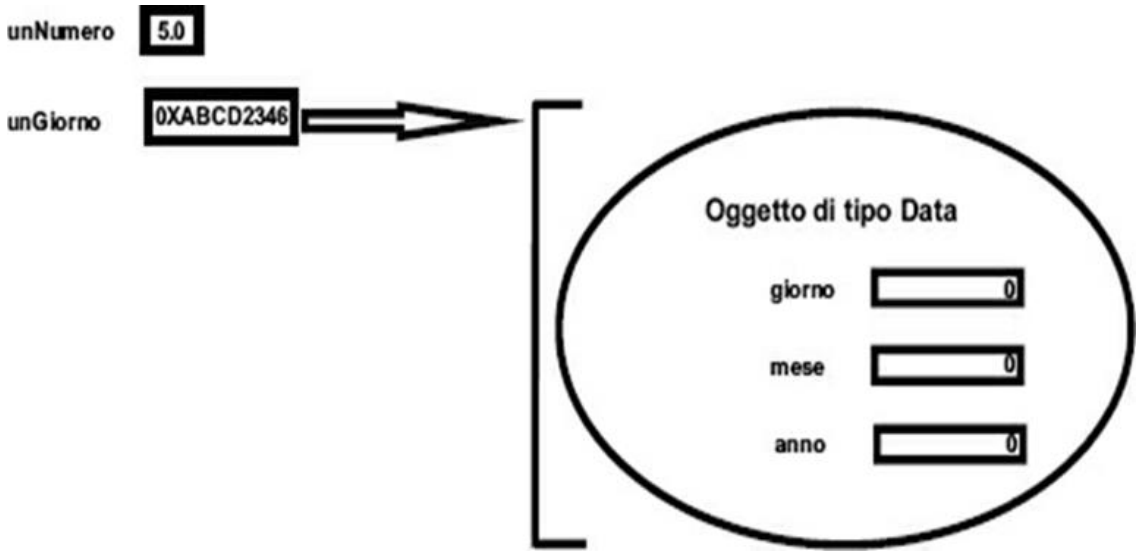



**Fig. 5: Convenzione di schematizzazione della memoria: dichiarazione di un tipo di dato intero (unNumero) e di un oggetto della classe Data**

La differenza pratica tra un reference ed una variabile diviene evidente se consideriamo le seguenti assegnazioni (Fig. 6). Sia per il tipo di dato primitivo che per quello non primitivo abbiamo creato un comportamente equivalente: dichiarazione ed assegnazione di un valore e dichiarazione di un dato dello stesso tipo con assegnazione dello stesso valore dell'altro.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

```
…
Double unNumero = 5.0;
Double unAltroNumero = unNumero;
Data unGiorno = new Data();
Data unAltroGiorno = unGiorno;
…
```

**Fig. 6: Esempio di assegnazioni nell'esempio considerato**

Come rappresentato in Fig. 7, la varabile unAltroNumero assumerà lo stesso valore della variabile unNumero ma le due variabili sono indipendenti e occupano celle differenti: sono l'una la copia dell'altra. Se il valore di una delle due variabili sarà modificato in seguito, il valore dell'altra variabile non cambierà.

Il reference unAltroGiorno invece assumerà il valore cioè l'indirizzo del reference unGiorno. Quindi se in seguito verrà apportata una qualche modifica all'oggetto comune tramite uno dei due reference, questa sarà verificabile anche tramite l'altro reference perché puntano alla stessa zona di memoria. Ovvero unGiorno.anno è equivalente a unAltroGiorno.anno.

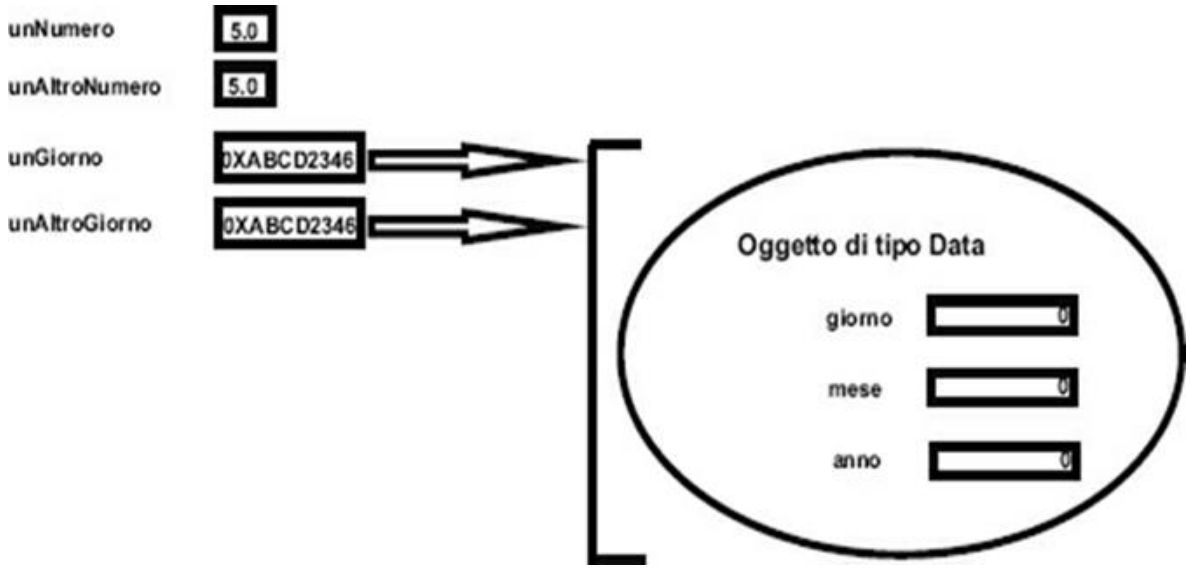



**Fig. 7: Convenzione di schematizzazione della memoria: assegnazioni**

In definitiva, una reference è il nome che diamo ad un oggetto, una particolare variabile che contiene come valore l'indirizzo a cui deve puntare e che deve conoscere il tipo che andrà a puntare. Questa è una differenza sostanziale che ha un impatto al momento del passaggio di parametri e dell'invocazione di un metodo. In Java quando si invoca un metodo che come parametro prende in input

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

una variabile, al metodo stesso viene passato il valore (una copia) di tale variabile che quindi rimane immutata anche dopo l'esecuzione del metodo.

Per esempio consideriamo la definizione della classe CiProvo di Fig. 8 con un metodo cambiaValore che cambia il valore di una variabile data in input. La classe CiProvo viene dichiarata e istanziata dopodiché una variabile denominata numero viene definita, inizializzata a 10 e passata la metodo cambiaValore.

```
…
public class CiProvo {
Public void cambiaValore (int valore) {
valore=1000;
}
}
…
CiProvo ogg = new CiProvo(); //dichiarazione e istanza dell'oggetto
                             //ogg di tipo CiProvo
int numero = 10;
ogg.cambiaValore(numero);    //chiamata al metodo cambiaValore
System.out.println ("il valore del numero è " + numero);
                                          //stampa di numero
…
```

**Fig. 8: classe CiProvo**

Dopodiché il valore di tale variabile numero viene stampato a video per verificarne il valore:

```
Il valore del numero è 10
```

L'output prodotto è 10, ovvero il valore della variabile numero non è cambiato. Infatti il parametro valore del motodo cambiaValore(), nel momento in cui è stato eseguito il metodo non coincideva con la variabile numero, bensì immagazzinava solo la copia del suo valore (10). Quindi naturalmente la variabile numero non è stata modificata.

Facciamo un esempio analogo con un oggetto anziché con una variabile di tipo intero.

Per esempio consideriamo la definizione della classe CiProvoConReference di Fig. 9 con un metodo cambiaReference che cambia il valore di un indirizzo che punta ad una certa area di memoria dato in ingresso con il valore di un indirizzo che punta ad un'altra area di memoria con i valori degli attributi inizializzati a zero (ottenuto tramite la new). La classe CiProvoConReference viene dichiarata e istanziata dopodiché una variabile denominata dataDiNascita di tipo Data viene definita, inizializzata a 14-4-2004 e passata la metodo cambiaReference, analogamente a prima.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

```
public class CiProvoConReference {
public void cambiaReference (Data data) {
data = new Data();
// Un oggetto appena istanziato
// ha le variabili inizializzate a valori nulli
}
}
…
CiProvoConReference ogg = new CiProvoConReference();
//dichiarazione e istanza dell'oggetto ogg di tipo CiProvoConReference
Data dataDiNascita = new Data();
dataDiNascita.giorno = 14;
dataDiNascita.mese = 4;
dataDiNascita.anno = 2004;
ogg.cambiaReference(dataDiNascita);
System.out.println("Data di nascita = " + dataDiNascita.giorno + " - " +
dataDiNascita.mese + " - " + dataDiNascita.anno);
…
```

**Fig. 8: classe CiProvoConReference**

Dopodiché il valore di tale oggetto dataDiNascita viene stampato a video per verificarne il valore.

```
Data di nascita = 14-4-2004
```

Analogamente a prima, al metodo viene sempre passato il valore del reference ma questo coincide con l'indirizzo in memoria dove risiede l'oggetto. Infatti al metodo CiProvoConReference è stata passata la copia del valore di un indirizzo che punta ad una certa area di memoria, poi questo valore viene cambiato con quello di un'altra area di memoria attraverso una new. Ma il valore iniziale passato non viene cambiato nel metodo e quindi a video viene stampato il valore inizialmente assegnato a dataDiNascita è rimasto inalterato.

Se però riscrivessimo il metodo cambiaReference() in modo tale che esso cambi i valori delle variabili d'istanza dell'oggetto, le cose cambierebbero. Consideriamo una nuova versione del metodo cambiaReference mostrata in Fig. 9.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

```
public class CiProvoConReference {
public void cambiaReference (Data data) {
data.giorno=12; //data punta allo stesso indirizzo
data.mese=11;   //della variabile dataDiNascita
data.anno=2006;
}
}
…
CiProvoConReference ogg = new CiProvoConReference();
Data dataDiNascita = new Data();
dataDiNascita.giorno = 14;
dataDiNascita.mese = 4;
dataDiNascita.anno = 2004;
ogg.cambiaReference(dataDiNascita);
System.out.println("Data di nascita = " + dataDiNascita.giorno + " - " +
dataDiNascita.mese + " - " + dataDiNascita.anno);

…
```

**Fig. 9: classe CiProvoConReference – seconda versione**

L'output cambierebbe:

```
Data di nascita = 12-11-2006
```

Infatti all'interno del metodo avremmo cambiato le variabili dell'oggetto originali, giacché il reference data (il parametro del metodo cambiaReference()) avrebbe puntato allo stesso indirizzo dell'oggetto in input (ovvero allo stesso indirizzo dell'oggetto). Il fatto che al metodo viene sempre passato il valore, garantisce che un oggetto possa essere modificato all'interno del metodo chiamato, e contemporaneamente, si è certi che dopo la chiamata del metodo il reference punti sempre allo stesso oggetto.

In altri linguaggi come il C, è permesso anche il passaggio di parametri per riferimento oltre a quello per valore. In quel caso al metodo viene passato l'intero reference, non solo l'indirizzo con la conseguente possibilità di poterne cambiare l'indirizzamento. Questa caratteristica non è stata importata da Java, perché considerata (a ragione) una minaccia alla sicurezza. Molti virus, work e simili, sfruttano infatti la tecnica del passaggio per riferimento. In questo casa Java scelse sin dall'inizio la strada della robustezza e della semplicità, favorendola rispetto alla potenza del linguaggio.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# Bibliografia


- [1] Il nuovo Java, Claudio De Sio Cesari, Hoepli Informatica


PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*